

숙박업소 예약 프로그램

C01팀

202510942 김강민
202513111 김도형
202211291 김호경
202411325 원승재
202511019 임서준
202411367 전재윤
202311403 한정원

목차

1. 문서 범위
2. 확장 목표
3. 추가 데이터 파일
 - 3.1 system.txt
 - 3.2 penalty.txt
 - 3.3 suspension.txt
4. 확장된 기존 데이터
 - 4.1 user.txt
 - 4.2 room.txt
 - 4.3 reservation.txt
 - 4.4 time.txt
 - 4.5 호텔 우편번호 변경과 예약 참조
5. 확장된 도메인 모델
 - 5.1 Reservation
 - 5.2 Role
 - 5.3 ReservationStatus
 - 5.4 DataSnapshot
6. 확장된 저장소 설계
7. 확장된 서비스 설계
 - 7.1 IntegrityService
 - 7.2 GuestService
 - 7.3 HostService
 - 7.4 PenaltyService
 - 7.5 SuspensionService
8. 관리자 기능
9. 고객 확장 기능
 - 9.1 체크인
 - 9.2 체크아웃
 - 9.3 리뷰
 - 9.4 예약 변경

10. 확장 입력 검증
11. 확장 오류 처리
12. 확장 구현 결정 사항

1. 문서 범위

이 문서는 1차 기획서 이후 확장 버전에서 새로 추가되거나 변경된 설계만 다룬다. 1차 기획서의 기본 로그인, 회원가입, 호텔/객실 등록, 단일 날짜 예약, 고객 취소 요청, 사장 취소 승인 흐름등은 반복해서 설명하지 않는다.

2. 확장 목표

확장 버전의 목표는 단일 날짜 예약 중심 구조를 체크인/체크아웃 날짜시간 기반 예약 시스템으로 넓히고, 패널티 파일 분리, 자동 영업정지, 리뷰, 관리자 기준 시각 변경 기능을 추가하는 것이다.

추가 구현 원칙은 다음과 같다.

1. 예약은 날짜 하나가 아니라 체크인 날짜/시간과 체크아웃 날짜/시간을 가진다.
2. 예약 상태는 체크인, 체크아웃, 취소 대기, 취소 완료, 리뷰 가능 상태를 표현할 수 있어야 한다.
3. 고객 패널티는 사용자 파일의 단일 필드가 아니라 별도 `penalty.txt`로 분리한다.
4. 숙소 영업정지는 별도 `suspension.txt`로 관리한다.
5. 관리자 역할을 추가하고 기준 시각 변경 기능을 제공한다.

확장 후 최종 데이터 파일은 `user.txt`, `hotel.txt`, `room.txt`, `reservation.txt`, `time.txt`, `system.txt`, `penalty.txt`, `suspension.txt` 총 8개다.

3. 추가 데이터 파일

3.1 system.txt

컬럼: 단일 `yyyy-MM` 또는 빈 값 `system.txt`는 일반 사용자 입력 날짜가 아니라 월별 자동 시스템 작업의 마지막 처리 월을 저장하는 내부 상태 파일이다. 따라서 날짜(연/월)의 일반 입력 문법과 별도로, 저장 형식은 `yyyy-MM`으로 정규화한다.

의미:

- 월 단위 자동 시스템 작업의 마지막 처리 월을 저장한다.
- 전월 1일 00:00부터 전월 말일 23:59까지 `CHECKED_OUT`이 완료되고 평점이 기록된 예약이 1건 이상 존재하는 호텔을 검사 대상으로 한다.
- 대상 예약들의 평균 평점에서 소수점 이하는 버림 처리하고, 그 값이 2.0 미만인 호텔은 14일, 즉 336시간 동안 영업정지된다.
- 자동 영업정지 기간과 겹치는 해당 호텔의 예약 중 상태가 `RESERVED` 또는 `CANCEL_PENDING`인 예약만 `CANCELLED`로 변경한다. 이때 `cancelRequestDate`에는

영업정지 시작일, 즉 해당 월 1일을 저장하고, `cancelRequestTime`에는 영업정지 시작 시각인 `00:00`을 저장한다. 이미 투숙 중인 `CHECKED_IN` 예약은 취소하지 않는다.

- 자동 영업정지로 취소된 예약은 고객 귀책 취소가 아니므로 최근 취소 횟수와 패널티 산정에 포함하지 않는다.
- 자동 영업정지 처리가 끝나면 처리 월을 `yyyy-MM` 형식으로 저장하여 같은 월 작업이 중복 실행되지 않게 한다.

읽는 시점:

- `FlatFileStore.loadSystem()`

쓰는 시점:

- `FlatFileStore.saveSystem()`
- 전체 저장 시 `FlatFileStore.saveAll()`
- 월별 자동 영업정지 처리 완료 시

도메인 객체:

- `SystemState`
 - `lastCheckedMonth : YearMonth?`
 - **getter/setter**
 - `verify(String...)`

서비스:

- `MonthlySystemService`
 - `runIfNeeded(LocalDateTime now)`
 - `findLowRatedHotels(YearMonth targetMonth)`
 - `grantAutomaticSuspension(Hotel hotel, LocalDateTime startAt, LocalDateTime endAt, String reason)`
 - `cancelReservationsOverlappingSuspension(Hotel hotel, LocalDateTime startAt, LocalDateTime endAt)`

3.2 penalty.txt

컬럼: `guestId`, `postalCode`, `endDate`

의미:

- 고객 패널티를 독립 파일로 관리한다.
- `postalCode`가 ALL이면 모든 호텔에 적용되는 전역 패널티다.
- `postalCode`가 ALL이 아니면 해당 우편번호 호텔에만 적용되는 호텔별 패널티다.
- 동일 고객에게 여러 패널티가 공존할 수 있다.

참조 관계:

- `guestId`는 `user.id` 중 GUEST 역할을 참조한다.
- `postalCode`가 ALL인 행은 호텔 참조 검사를 하지 않는다.
- `postalCode`가 ALL이 아닌 행은 `hotel.postalCode`를 참조한다.

활성 조건:

- `today <= endDate`

도메인 객체:

- `Penalty`
 - `guestId` final String
 - `postalCode` final String
 - `endDate` final LocalDate
 - `isGlobal()`은 `postalCode.equals("ALL")`일 때 true를 반환한다.
 - `isActiveOn(LocalDate)`
 - `affects(String postalCode)`
 - `verify(String...)`

서비스:

- `PenaltyService`
 - `hasActiveGlobalPenalty(User user, LocalDate today)`
 - `hasActivePenaltyOn(User user, String postalCode, LocalDate today)`
 - `activePenaltiesOf(User user, LocalDate today)`
 - `grantGlobal(User user, LocalDate endDate)`는 `postalCode`에 ALL을 저장한다.
 - `grantHotel(User user, String postalCode, LocalDate endDate)`
 - `purgeExpired(LocalDate today)`

3.3 suspension.txt

컬럼: `postalCode`, `startAt`, `endAt`, `reason`

의미:

- 호텔 영업정지 기간을 관리한다.
- 같은 호텔에 여러 정지 구간이 공존할 수 있다.
- `reason`에는 정지 사유를 저장한다.
- 월간 평점 2.0 미만으로 자동 영업정지된 경우 `reason`은 `LOW_RATING_MONTHLY`로 저장한다.

참조 관계:

- `postalCode`는 `hotel.postalCode`를 참조한다.

활성 조건:

- `startAt <= now < endAt`

구간 중첩 조건:

- 두 기간의 실제 영업정지 시간이 겹치면 중첩으로 본다.

도메인 객체:

- Suspension
 - postalCode final String
 - startAt final LocalDateTime
 - endAt final LocalDateTime
 - reason final String
 - isActiveAt(LocalDateTime)
 - overlaps(LocalDateTime, LocalDateTime)
 - verify(String...)

저장소 기능:

- FlatFileStore.loadSuspensions()
- FlatFileStore.saveSuspensions()
- FlatFileStore.suspensions()

4. 확장된 기존 데이터

4.1 user.txt

최종 컬럼:

- id, password, name, role

추가/변경 사항:

- role 값에 admin을 추가한다.
- 1차 기획서의 고객 패널티 종료일 필드는 제거한다.
- 패널티는 penalty.txt에서 관리한다.

추가 무결성:

- 호텔 소유자는 HOST여야 한다.
- 예약 고객과 패널티 대상은 GUEST여야 한다.
- 관리자는 관리자 메뉴 진입에만 사용한다.

4.2 room.txt

변경 사항:

- 객실의 기본 체크인/체크아웃 시간은 예약 입력의 기본 안내값으로 사용한다.
- checkInTime == checkOutTime은 허용하지 않는다.
- checkInTime > checkOutTime은 허용하지 않는다.
- 객실 기본 시간만으로 익일 체크아웃을 자동 해석하지 않는다.
- 실제 예약의 체크인/체크아웃은 reservation.txt의 체크인 날짜시간과 체크아웃 날짜시간을 기준으로 판단한다.

4.3 reservation.txt

최종 컬럼:

- guestId
- hotelName
- postalCode
- roomNumber
- guestCount
- checkInDate
- checkInTime
- checkOutDate
- checkOutTime
- status
- createdAt
- cancelRequestDate
- cancelRequestTime
- rating

확장 의미:

- hotelName은 예약 당시 호텔명을 보존하는 스냅샷 필드다.
- 호텔명 변경 시 기존 예약의 hotelName은 변경하지 않는다.
- postalCode도 예약 당시 참조 필드로 취급한다.
- 호텔 우편번호 변경 시 기존 예약의 postalCode는 변경하지 않는다.
- 객실 번호 변경 시 기존 예약의 roomNumber는 변경하지 않는다.
- 명시적 예약 ID는 추가하지 않는다. UI에서 목록을 보여준 뒤 사용자가 선택한 리스트 인스턴스를 수정 대상으로 사용한다.

확장 상태:

- RESERVED: 예약됨
- CANCEL_PENDING: 취소 승인 대기
- CANCELLED: 취소됨
- CHECKED_IN: 입실
- CHECKED_OUT: 퇴실

상태별 필드 규칙:

- RESERVED 상태시는 cancelRequestDate와 cancelRequestTime이 비어 있어야 한다.
- CANCEL_PENDING 상태시에는 cancelRequestDate와 cancelRequestTime이 있어야 한다.
- cancelRequestDate와 cancelRequestTime은 취소 요청 이력 표시용이며, CANCEL_PENDING 자동 복구 기준으로 사용하지 않는다.
- CANCELLED 예약의 cancelRequestDate와 cancelRequestTime은 취소 원인에 따라 다음과 같이 저장한다.
 - 고객 취소 요청으로 취소된 경우 고객이 취소를 요청한 시점의 기준 날짜와 시간을 저장한다.

- 체크인까지 4일 이상 남아 고객 요청 즉시 CANCELLED가 된 경우, 즉시 취소 처리 시점의 기준 날짜와 시간을 저장한다.
- CANCEL_PENDING 상태였던 예약이 사장 승인으로 CANCELLED가 된 경우, CANCEL_PENDING으로 변경될 때 저장된 cancelRequestDate와 cancelRequestTime을 그대로 유지한다.
- 자동 영업정지로 취소된 경우 cancelRequestDate에는 영업정지 시작일, 즉 해당 월 1일을 저장하고, cancelRequestTime에는 영업정지 시작 시각인 00:00을 저장한다.
- 자동 영업정지로 취소된 예약은 고객 귀책 취소가 아니므로 최근 취소 횟수와 패널티 산정에 포함하지 않는다.
- CHECKED_OUT은 rating을 가질 수 있다.
- CHECKED_OUT이 아닌 상태는 rating이 비어 있어야 한다.

시간 규칙:

- createdAt의 저장 형식은 yyyy-MM-dd HH:mm이다. cancelRequestDate는 yyyy-MM-dd, cancelRequestTime은 HH:mm으로 저장한다.
- (checkOutDate, checkOutTime)은 (checkInDate, checkInTime)보다 반드시 늦어야 한다.
- 예약 시간 중복은 같은 객실의 기존 차단 예약과 실제 투숙 시간이 겹치는 경우로 판단한다.
- CANCELLED를 제외한 모든 예약은 시간 중복 검사에서 차단 예약이다.

4.4 time.txt

변경 사항:

- 1차 기획서의 기준 날짜를 기준 시각으로 확장한다.
- 파일은 단일 문자열만 가진다.
- 허용 형식은 날짜시간, 날짜 단독, 빈 값이다.
- 날짜 단독이면 해당 날짜의 00:00으로 해석한다.
- 빈 값이거나 파싱할 수 없으면 Asia/Seoul 기준 현재 실시간으로 보정한다.
- 저장 형식은 yyyy-MM-dd HH:mm이다.

4.5 호텔 우편번호 변경과 예약 참조

확장 버전에서는 예약의 postalCode를 예약 당시 숙소 위치를 보여주는 스냅샷 값으로 취급한다.

처리 정책:

- 호텔 우편번호 변경 시 hotel.txt와 해당 호텔의 room.txt 우편번호만 변경한다.
- 기존 예약의 postalCode는 변경하지 않는다.
- 취소되지 않은 활성 예약이 남아 있어도 호텔 우편번호 변경 자체를 차단하지 않는다.
- CANCELLED, CHECKED_OUT 예약뿐 아니라 활성 예약도 예약 당시의 hotelName, postalCode, roomNumber를 과거 내역 표시용 스냅샷으로 유지한다.

- 우편번호 변경 후 새 예약과 객실 조회는 변경된 현재 호텔/객실 정보를 기준으로 처리한다.

5. 확장된 도메인 모델

5.1 Reservation

추가 필드:

- `hotelName : String`
- `checkInDate : LocalDate`
- `checkInTime : LocalTime`
- `checkOutDate : LocalDate`
- `checkOutTime : LocalTime`
- `createdAt : LocalDateTime`
- `rating : Double?`

추가 동작:

- `isBlockingReservation()` 은 CANCELLED를 제외한 모든 상태에서 `true`를 반환한다.
- 체크인, 체크아웃, 리뷰 작성, 예약 변경 과정에서 기존 예약 인스턴스의 필드를 직접 수정한다.

5.2 Role

추가 값:

- `ADMIN("admin", "관리자")`

역할 분기:

- 로그인 성공 후 CUSTOMER, OWNER, ADMIN 메뉴로 분기한다.

5.3 ReservationStatus

변경 값:

- `RESERVED`
- `CANCEL_PENDING`
- `CANCELLED`
- `CHECKED_IN`
- `CHECKED_OUT`

상태 전이:

- 예약 생성: `RESERVED`

- 체크인: RESERVED -> CHECKED_IN
- 체크아웃: CHECKED_IN -> CHECKED_OUT
- 취소 요청: RESERVED -> CANCEL_PENDING 또는 RESERVED -> CANCELLED
- 사장 취소 승인: CANCEL_PENDING -> CANCELLED
- 사장 취소 거절: CANCEL_PENDING -> RESERVED

5.4 DataSnapshot

추가 필드:

- List<Penalty> penalties
- List<Suspension> suspensions
- SystemState systemState
- LocalDateTime baselineDateTime

변경 사항:

- 기준일은 LocalDate 단속이 아니라 LocalDateTime으로 보관한다.
- baselineDate()는 baselineDateTime.toLocalDate()를 통해 제공한다.

6. 확장된 저장소 설계

FlatFileStore는 8개 데이터 파일을 관리한다.

최종 관리 파일:

- user.txt
- hotel.txt
- room.txt
- reservation.txt
- time.txt
- system.txt
- penalty.txt
- suspension.txt

추가 파일:

- system.txt
- penalty.txt
- suspension.txt

추가 공개 인터페이스:

- penalties()
- suspensions()
- systemState()
- baselineDateTime()
- setBaselineDateTime(LocalDateTime)

- `penaltyFile()`
- `suspensionFile()`
- `systemFile()`
- `loadPenalties()`
- `loadSuspensions()`
- `loadSystem()`
- `savePenalties()`
- `saveSuspensions()`
- `saveSystem()`

초기화 시 추가 처리:

1. 8개 파일의 존재 여부와 권한을 확인한다.
2. `time.txt`에서 기준 시각을 읽는다.
3. 패널티, 영업정지, 시스템 상태를 함께 로드한다.
4. 패널티와 영업정지의 참조 무결성을 검사한다.
5. 만료된 패널티와 영업정지를 삭제한다.
6. `CANCEL_PENDING` 예약 중 현재 기준 시각이 체크인 시각의 24시간 전에 도달했는데도 사장 승인/거절이 처리되지 않은 예약은 `RESERVED`로 복구하고 `cancelRequestDate`와 `cancelRequestTime`을 비운다.
7. 필요한 경우 월별 자동 영업정지 검사를 수행한다.
8. 정규화된 전체 데이터를 다시 저장한다.

7. 확장된 서비스 설계

7.1 IntegrityService

추가 책임:

- 확장된 `user.txt`, `room.txt`, `reservation.txt`, `time.txt` 문법 검증
- `penalty.txt` 문법 검증
- `suspension.txt` 문법 검증
- `system.txt` 문법 검증
- 확장 예약 상태와 상태별 필드 규칙 검증
- 패널티 참조 무결성 검증
- 영업정지 참조 무결성 검증
- 만료 패널티 삭제
- 만료 영업정지 삭제

예약 참조 검증:

- 모든 예약의 고객은 현재 존재하는 `GUEST` 사용자여야 한다.
- 예약의 `hotelName`, `postalCode`, `roomNumber`는 예약 당시 스냅샷으로 보존될 수 있다.
- 호텔명, 우편번호, 객실 번호 변경 후 기존 예약이 현재 호텔/객실 값과 달라져도 그 자체를 치명 오류로 처리하지 않는다.
- `CANCELLED`가 아닌 예약끼리는 같은 객실에서 실제 투숙 시간이 겹칠 수 없다.

만료 및 자동 복구 규칙:

- 패널티는 `today > endDate`이면 삭제한다.
- 영업정지는 `now >= endAt`이면 삭제한다.
- `CANCEL_PENDING` 예약은 `now >= checkInDateTime.minusHours(24)`이면 자동으로 `RESERVED`로 복구하고 `cancelRequestDate`와 `cancelRequestTime`을 비운다. 복구 기준은 취소 요청 발생 시각이 아니라 예약된 체크인 시각이다.

7.2 GuestService

추가 기능:

- 영업 중인 호텔 목록 조회
- 영업 중인 호텔 검색
- 전역 패널티와 호텔별 패널티에 따른 예약 제한
- 예약 기간과 영업정지 기간 중첩 검사
- 예약 시간 중복 검사
- 체크인/체크아웃 시각 기반 예약 생성
- 기존 예약 변경
- 체크인, 체크아웃, 리뷰 작성에 필요한 예약 상태 갱신 저장
- 체크인/체크아웃 시각 패널티 부여
- 체크인 24시간 전까지 미처리된 취소 요청 자동 복구

추가 인터페이스:

- `listOpenHotels(LocalDateTime now)`
- `searchOpenHotels(String keyword, LocalDateTime now)`
- `isHotelSuspendedAt(String postalCode, LocalDateTime now)`
- `isHotelSuspensionOverlapping(String postalCode, LocalDateTime from, LocalDateTime to)`
- `isDateBlocked(String postalCode, int roomNumber, LocalDate date, LocalDate baselineDate)`
- `isTimeOverlapping(String postalCode, int roomNumber, LocalDateTime newStart, LocalDateTime newEnd, Reservation exclude)`
- `replaceReservation(...)`
- `updateReservation(Reservation reservation)`
- `restoreExpiredCancelPending(LocalDateTime now)`
- `shouldRestoreCancelPending(Reservation reservation, LocalDateTime now)`

예약 생성 추가 규칙:

- 전역 패널티가 활성화 상태인 고객은 어떤 호텔도 예약할 수 없다.
- 호텔별 패널티가 활성화 상태인 고객은 해당 호텔을 예약할 수 없다.
- 예약 시작 날짜는 현재 기준 날짜보다 이후여야 한다. 즉, 당일 예약은 불가능하다.
- 예약 시작 시각은 현재 기준 시각보다 이후여야 한다.
- 예약 인원은 객실 정원 이하여야 한다.
- 체크아웃 날짜시간은 체크인 날짜시간보다 반드시 늦어야 한다.

- 객실 기본 체크인/체크아웃 시간만으로 익일 체크아웃을 자동 결정하지 않는다.
- 영업정지 기간과 겹치면 예약할 수 없다.
- 이미 점유된 시간 구간과 겹치면 예약할 수 없다.

취소 요청 확장 규칙:

- 취소 대상은 RESERVED 상태 예약만 허용한다.
- 현재 기준 시각이 체크인 시각 이상이면 취소할 수 없다.
- 최근 7일 안에 CANCEL_PENDING 또는 CANCELLED가 된 예약이 3개를 초과하면 전역 패널티를 14일 부여하고 대상 예약 상태는 변경하지 않는다.
- 체크인 날짜까지 3일 이하로 남았으면 상태를 CANCEL_PENDING으로 바꾸고 cancelRequestDate에 현재 기준 날짜를, cancelRequestTime에 현재 기준 시간을 저장한 뒤 사장 승인을 기다린다.
- 체크인 날짜까지 4일 이상 남았으면 상태를 즉시 CANCELLED로 바꾸고 cancelRequestDate에 현재 기준 날짜를, cancelRequestTime에 현재 기준 시간을 저장한다.
- 남은 일수는 `DAYS.between(today, checkInDate)`로 계산한다. 단, 체크인 당일이거나 이미 체크인 시각이 지난 경우는 취소 불가가 우선한다.
- CANCEL_PENDING 예약은 예약된 체크인 시각으로부터 정확히 24시간 전까지 사장 승인 또는 거절이 처리되지 않으면 RESERVED로 자동 복구되고 cancelRequestDate와 cancelRequestTime은 비워진다.
- 자동 복구 기준은 취소 요청 저장값인 cancelRequestDate와 cancelRequestTime으로부터 24시간 경과가 아니라 `checkInDateTime.minusHours(24)` 도달 여부다.

고객 안내 문구:

- 예약 변경 시 기존 예약의 체크인 날짜와 현재 기준 날짜 차이가 3일 이내라서 호텔별 3일 패널티가 부여되는 경우, 다음 문구를 출력한다: 예약 날짜로부터 3일 이내에 예약을 변경하여 해당 업소 한정 3일간 예약 불가 페널티를 부여합니다.
- 최근 7일 안에 CANCEL_PENDING 또는 CANCELLED가 된 예약이 3개를 초과하여 전역 14일 패널티가 부여되는 경우, 전역 패널티 부여 사실과 14일 동안 전체 업소 예약이 제한된다는 전용 안내 문구를 출력한다.

예약 변경 패널티 규칙:

- 기존 예약의 체크인 날짜와 현재 기준 날짜의 차이가 3일 이내인 상태에서 예약을 변경하면 해당 업소 한정 3일 예약 불가 패널티를 부여한다.
- 예약 변경 패널티는 호텔별 패널티이며 `penalty.txt`의 `postalCode`에는 변경 전 예약의 `postalCode`를 저장한다.
- 패널티 종료일은 현재 기준 날짜로부터 3일 후로 저장한다.

지각 패널티 규칙:

- 체크인 처리 시 현재 기준 시각이 예약 체크인 시각보다 1시간 이상 늦으면 해당 호텔에 대한 1주일 예약 불가 패널티를 부여한다.
- 체크아웃 처리 시 현재 기준 시각이 예약 체크아웃 시각보다 1시간 이상 늦으면 해당 호텔에 대한 1주일 예약 불가 패널티를 부여한다.
- 지각 패널티는 호텔별 패널티이며 `penalty.txt`의 `postalCode`에는 해당 예약의 `postalCode`를 저장한다.

7.3 HostService

추가/변경 기능:

- 호텔명 변경 시 기존 예약의 `hotelName`은 변경하지 않는다.
- 호텔 우편번호 변경 시 기존 예약의 `postalCode`는 변경하지 않는다.
- 호텔 우편번호 변경은 현재 활성 예약 존재 여부와 무관하게 허용하되, 변경 후 `hotel.txt`와 해당 호텔의 `room.txt`만 갱신한다.
- 객실 번호 변경 시 기존 예약의 `roomNumber`는 변경하지 않는다.
- 취소 승인 대상은 `CANCEL_PENDING` 상태 예약이다.
- 사장이 취소 승인/거절 목록에서 예약을 선택한 직후에도 먼저 자동 복구 규칙을 재검사한다.
- 선택한 예약이 체크인 **24시간** 전 자동 복구 규칙으로 이미 `RESERVED`가 되었거나 처음부터 `CANCEL_PENDING`이 아니면 처리를 중단하고 취소 요청 대기 상태의 예약만 처리할 수 있습니다.를 출력한다.

취소 승인/거절 규칙:

- 승인/거절 처리 직전에 해당 예약의 자동 복구 여부를 다시 검사한다.
- 승인하면 상태를 `CANCELLED`로 바꾸고 `cancelRequestDate`와 `cancelRequestTime`은 유지한다.
- 거절하면 상태를 `RESERVED`로 되돌리고 `cancelRequestDate`와 `cancelRequestTime`을 비운다.
- 자동 복구 또는 다른 처리로 인해 선택 예약이 더 이상 `CANCEL_PENDING`이 아니면 승인/거절을 수행하지 않고 취소 요청 대기 상태의 예약만 처리할 수 있습니다.를 출력한다.
- 승인 또는 거절 후 `reservation.txt`를 저장한다.

7.4 PenaltyService

새 서비스다.

책임:

- 전역 패널티 조회
- 호텔별 패널티 조회
- 사용자별 활성 패널티 목록 조회
- 전역 패널티 부여
- 호텔별 패널티 부여
- 만료 패널티 삭제

저장 부수효과:

- 패널티 추가나 삭제가 발생하면 `penalty.txt`를 저장한다.

7.5 SuspensionService

새 서비스다.

책임:

- 호텔별 영업정지 목록 조회
- 월별 자동 영업정지 등록
- 만료 영업정지 삭제
- 영업정지와 겹치는 예약 자동 취소

인터페이스:

- `suspensionsOf(String postalCode)`
- `grantAutomatic(String postalCode, LocalDateTime startAt, LocalDateTime endAt, String reason)`
- `purgeExpired(LocalDateTime now)`
- `cancelOverlappingReservations(String postalCode, LocalDateTime startAt, LocalDateTime endAt)`

등록 규칙:

- `startAt`은 `endAt`보다 이전이어야 한다.
- `postalCode`는 현재 존재하는 호텔을 참조해야 한다.
- 같은 호텔에 여러 영업정지 기간이 공존할 수 있다.
- 자동 영업정지 등록 시 해당 기간과 겹치는 예약 중 **RESERVED**, **CANCEL_PENDING** 상태의 예약만 **CANCELLED**로 변경한다. **CHECKED_IN** 예약은 이미 투숙 중인 예약이므로 자동 영업정지로 취소하지 않는다.
- 자동 영업정지로 취소된 예약은 `cancelRequestDate`에 영업정지 시작일, `cancelRequestTime`에 영업정지 시작 시각 00:00을 저장하며, 고객 패널티 계산에는 포함하지 않는다.
- 등록 또는 삭제가 발생하면 `suspension.txt`를 저장한다.

8. 관리자 기능

추가 역할:

- ADMIN

관리자 메뉴:

- 기준 시각 변경
- 로그아웃

기준 시각 변경:

- 사용자가 입력한 날짜 또는 날짜시간을 파싱한다.
- `FlatFileStore.setBaselineDateTime(...)`으로 메모리 기준 시각을 변경한다.
- `time.txt`를 저장한다.
- 기준 시각 변경 직후 만료 패널티와 만료 영업정지를 정리하고 변경된 파일을 저장한다.

- 기존 시각 변경 직후 예약된 체크인 시각의 **24시간** 전까지 처리되지 않은 CANCEL_PENDING 예약을 RESERVED로 복구한다.
- 기존 시각 변경 직후 필요한 경우 월별 자동 영업정지 검사를 수행한다.
- 날짜만 입력하면 해당 날짜의 00:00으로 저장한다.

영업정지 처리:

- 영업정지는 관리자가 수동으로 등록하거나 해제하지 않는다.
- 영업정지는 매월 1일 자동 시스템 작업으로만 등록된다.
- 관리자 메뉴에는 영업정지 등록/해제 항목을 두지 않는다.

9. 고객 확장 기능

9.1 체크인

조건:

- 로그인 고객 본인의 예약
- 예약 상태가 RESERVED
- 현재 기준 시각이 예약 체크인 시각 이후

처리:

- 현재 기준 시각이 예약 체크인 시각보다 1시간 이상 늦으면 해당 호텔에 대한 1주일 예약 불가 패널티를 부여한다.
- 예약 상태를 CHECKED_IN으로 변경한다.
- reservation.txt를 저장한다.
- 패널티가 부여된 경우 penalty.txt도 저장한다.

9.2 체크아웃

조건:

- 로그인 고객 본인의 예약
- 예약 상태가 CHECKED_IN
- 현재 기준 시각이 예약 체크아웃 시각 이후

처리:

- 현재 기준 시각이 예약 체크아웃 시각보다 1시간 이상 늦으면 해당 호텔에 대한 1주일 예약 불가 패널티를 부여한다.
- 예약 상태를 CHECKED_OUT으로 변경한다.
- reservation.txt를 저장한다.
- 패널티가 부여된 경우 penalty.txt도 저장한다.

9.3 리뷰

조건:

- 로그인 고객 본인의 예약
- 예약 상태가 CHECKED_OUT
- 아직 평점이 없음

처리:

- 0.0 이상 10.0 이하의 유리수 평점을 저장한다.
- reservation.txt를 저장한다.

9.4 예약 변경

조건:

- 로그인 고객 본인의 예약
- 변경 대상 예약 상태가 RESERVED여야 한다.
- 현재 기준 시각이 기존 체크인 시각보다 이전이어야 한다.
- 새 예약 시작 날짜는 현재 기준 날짜보다 이후여야 한다. 즉, 당일 예약은 불가능하다.
- 새 예약 시작 시각은 현재 기준 시각보다 이후여야 한다.
- 새 예약 기간이 영업정지와 겹치지 않아야 한다.
- 새 예약 기간이 다른 차단 예약과 겹치지 않아야 한다.

처리:

- 기존 예약의 체크인 날짜와 현재 기준 날짜 차이가 3일 이내이면 해당 업소 한정 3일 예약 불가 패널티를 부여하고, 예약 날짜로부터 3일 이내에 예약을 변경하여 해당 업소 한정 3일간 예약 불가 패널티를 부여합니다.를 출력한다.
- 기존 예약 인스턴스의 호텔, 객실, 인원, 체크인/체크아웃 필드를 수정한다.
- 상태는 RESERVED로 유지한다.
- 취소 요청 날짜/시간과 평점은 제거한다.
- reservation.txt를 저장한다.
- 패널티가 부여된 경우 penalty.txt도 저장한다.

10. 확장 입력 검증

추가 검증:

- 평점: 0.0 이상 10.0 이하의 유리수만 허용
- 날짜시간: 날짜와 시간이 표준 공백 하나로 분리된 yyyy-MM-dd HH:mm 형태로 저장
- 예약 상태: RESERVED, CANCEL_PENDING, CANCELLED, CHECKED_IN, CHECKED_OUT
- 역할 저장값: host, guest, admin만 허용
- 역할 사용자 입력값은 대소문자 입력을 받을 수 있으나 저장 전 소문자로 정규화한다.
- penalty.txt의 전역 패널티 우편번호: ALL만 허용

- 날짜(연/월)의 일반 문법은 기획서 4.5.2를 따른다. 다만 **system.txt**는 월별 자동 시스템 작업의 중복 실행 방지를 위한 내부 상태 파일이므로, 기획서 5.4의 예외 규칙에 따라 저장 형식을 **yyyy-MM**으로 정규화한다.

시간 파서:

- HH:mm, HH/mm, HH.mm, HH-mm
- AM/PM 표기 허용
- 저장 형식은 HH:mm

날짜시간 저장 형식:

- yyyy-MM-dd HH:mm

평점 저장 형식:

- 0 이상 10 이하의 유리수 값을 문자열로 저장한다.
- 0.5 단위 제한은 두지 않는다.

11. 확장 오류 처리

추가 예외 구분:

- `FatalDataException`: 복구 불가능한 데이터/무결성 오류
- `RecoverableInputException`: 사용자가 다시 입력할 수 있는 UI 입력 오류

상태 그래프 처리:

- `RecoverableInputException`은 메시지를 `RuntimeContext`의 **pending error**에 저장한다.
- 이후 현재 상태를 다시 실행해 같은 화면에서 재입력하도록 한다.

12. 확장 구현 결정 사항

- 데이터 파일은 계속 탭 구분 텍스트 파일을 사용한다.
- 저장은 계속 파일 전체 덮어쓰기 방식이다.
- 예약 시간 중복은 같은 객실에서 실제 투숙 시간이 겹치는 경우로 판단한다.
- 예약 날짜 달력 차단은 날짜 단위 양 끝 포함 규칙을 사용한다.
- 호텔명 변경은 기존 예약의 `hotelName`에 반영하지 않는다.
- 호텔 우편번호 변경은 기존 예약의 `postalCode`에 반영하지 않는다.
- 객실 번호 변경은 기존 예약의 `roomNumber`에 반영하지 않는다.
- `time.txt`가 비어 있거나 깨졌을 때 프로그램을 실패시키지 않고 **KST** 현재 실시간으로 보정한다.
- 전역 패널티는 `penalty.txt`의 `postalCode` 값 ALL로 표현한다.
- 자동 영업정지 사유는 `suspension.txt`의 `reason` 컬럼에 저장한다.
- 매월 1일 자동 영업정지로 취소된 예약은 고객의 최근 취소 횟수와 패널티 산정에 포함하지 않는다.
- `CANCEL_PENDING` 예약은 예약된 체크인 시각의 **24시간** 전까지 사장 승인 또는 거절이 처리되지 않으면 자동으로 `RESERVED`로 복구한다.

- Penalty와 Suspension은 불변 객체로 취급한다.